

Практическое задание по работе с репозиторием Git

Задание заключается в отработке навыков командной разработки на языке C++ при работе с Git

Студентам предлагается клонировать репозиторий, создать отдельную ветку для внесения изменений в исходный файл программы в соответствии с заданием. Необходимо проверить что после внесённых изменений программа корректно компилируется и выполняется.

После этого необходимо залить ветку в репозиторий и создать запрос на слияние в ветку с результатом.

Задание сделано таким образом, что при слиянии в ветку с результатом должен возникнуть конфликт.

Студенту необходимо разрешить конфликт и обновить запрос на слияние.

Успешным выполнением лабораторной работы будет считаться заливка созданных студентом изменений в ветку с результатом.

Перед выполнением задания необходимо выполнить следующее:

- Установить Git
- Установить и настроить GitExtensions (визуальная оболочка для работы с Git)
- Установить и настроить WinMerge (редактор для разрешения конфликтов)
- Настроить Visual Studio Code для редактирования и компиляции C++
- Зарегистрироваться на сайте GitVerse
- Изучить основы работы с репозиториями и команды Git прочитав файл «Лабораторная работа Git (github).docx»
- Узнать у преподавателя ссылку на репозиторий с заданием
- Клонировать репозиторий на локальную машину (можно клонировать с одновременным переключением на ветку «task»)
- Проверить имя текущей ветки. Если это не «task», то переключиться на ветку «task»

Описание задания

Задание заключается в доработке файла GroupList.cpp

В зависимости от выбранного преподавателем задания, содержимое файла может различаться. Это не влияет на те изменения, которые нужно сделать.

Код в файле предназначен для создания списка группы и вывода результата на экран. С кодом необходимо ознакомиться самостоятельно.

Краткое описание ключевых моментов из кода файла:

- Структура People описывает информацию о человеке для отображения
- Метод showPeoples отображает информацию из списка структур People
- В методе main производится создание списка структур People и вызывается метод showPeoples для отображения списка на консоли/в терминале

Действия после первоначального клонирования репозитория

1. После клонирования репозитория и до начала выполнения задания, необходимо создать отдельную локальную ветку Git, в которой будет производиться доработка файлов

согласно заданию.

!!! Локальная ветка, для внесения изменений должна быть создана из ветки task. Для этого, перед созданием ветки необходимо убедиться что текущая активная ветка task

!!! Имя локальной ветки необходимо сформировать по следующему шаблону:

<порядковый номер в журнале с ведущим нулём, если он меньше десяти>_<фамилия и имя на английском(разделитель – знак подчёркивания)>

Например: 02_Borisov_Ivan

После выполнения задания нужно будет зафиксировать изменения в ветке (сделать commit) и передать ветку на сервер, где сделать merge request, чтобы залить изменения в результирующую ветку (result);

2. Изменение файла «GroupList.cpp»:
 - a. В файл, вместо комментария «// <<< Сюда необходимо добавить функции создания структур с информацией о студенте» необходимо добавить код функции, которая должна создать, заполнить и вернуть структуру People с информацией о студенте. Дополнительно, необходимо выполнить следующие условия:
 - i. Имя метода должно начинаться с create. Далее нужно написать порядковый номер в журнале с ведущим нулём, если он меньше десяти и фамилию студента

Например:

```
People create02Borisov()
{
    People student;

    student.id = 2;
    student.name = "Борисов Иван";
    student.height = 176;

    return student;
}
```

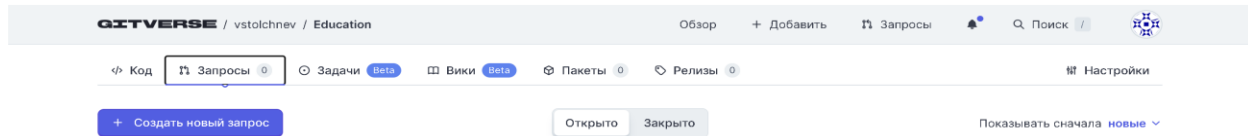
- b. В метод main, вместо комментария «// <<< Сюда добавить вызов функций создания структур с информацией о студентах // (необходимо соблюдать порядок по возрастанию идентификаторов - порядковому номеру в журнале)» необходимо добавить код с вызовом добавленного метода и добавлением результата в список

Например:

```
listOfPeoples.push_back(create02Borisov());
```

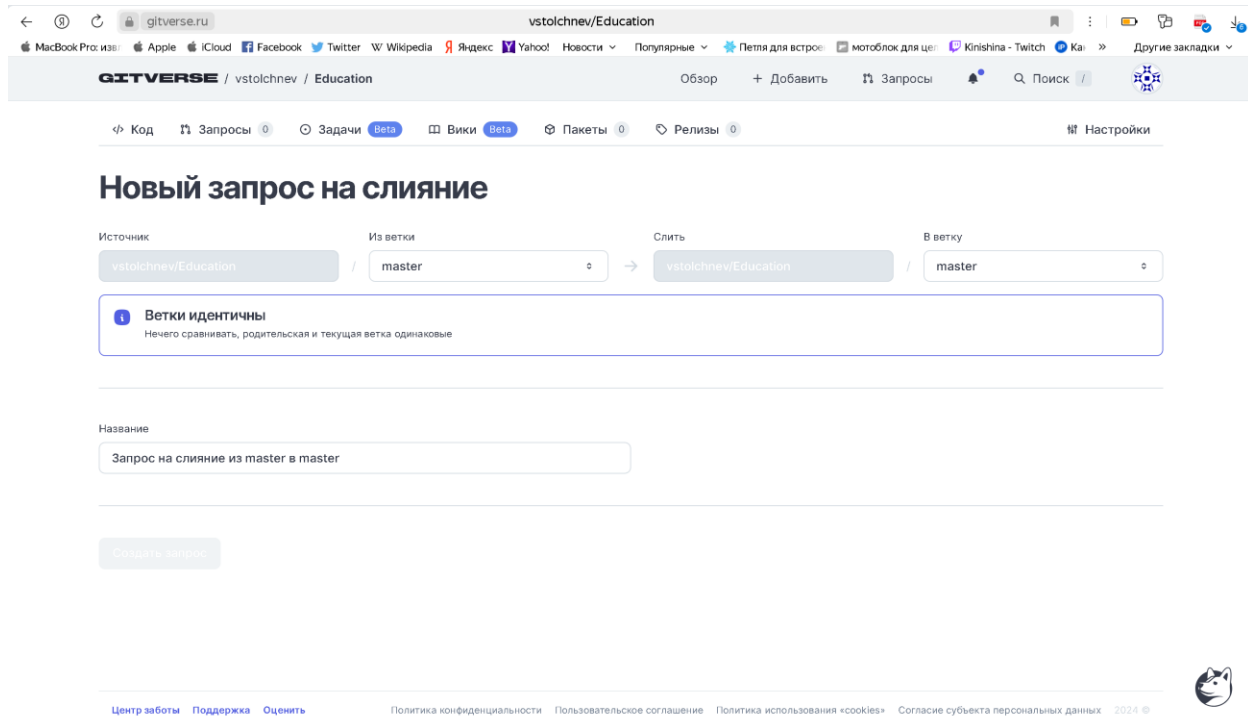
3. После сделанных изменений необходимо проверить скомпилировать и проверить работу программы
4. Если всё работает корректно, то необходимо зафиксировать изменения, но
 - a. **!!! Важно!!! Перед фиксацией изменений необходимо проверить что ваш коммит не удаляет изменения сделанные другими студентами;**
 - b. (сделать commit) в ранее созданную ветку и передать ветку на сервер (сделать push)

5. На сервере необходимо сделать для своей ветки «Merge Request» (Запрос на слияние)



(при создании необходимо назначить «Merge Request» на преподавателя).

!!! Merge Request необходимо делать на ветку result



6. После создания MergeRequest необходимо убедиться что в нём нет конфликтов. Если конфликт есть, то необходимо дождаться, когда MergeRequest станет первым на обработку в списке и перейти к пункту, где описывается процедура исправления конфликта.
7. После принятия «Merge Request» преподавателем задание считается выполненным

Действия при возникновении конфликта при попытке залить изменения в основную ветку через «Merge Request»

При возникновении конфликта необходимо выполнить следующие действия:

1. Перейти в локальном репозитории в основную ветку («result»)
2. Загрузить изменения с сервера (актуализировать ветку), выполнив команду Pull
3. Перейти в свою ветку
4. Запустить процесс слияния ветки «result» со своей веткой.
5. Процесс слияния прервётся на том, что в файле GroupList.cpp будут найдены конфликты. Конфликт необходимо разрешить, используя редактор конфликтов (программа Kdiff3 в GitExtensions).
6. В появившемся редакторе конфликтов необходимо в нижнем окне редактирования получить итоговый вариант (разрешить конфликт):
Конфликт будет заключаться в том, что другие студенты будут в том же месте прописывать

свой код для отображения информации о себе. Файл необходимо модифицировать таким образом, чтобы не затереть добавленное другими студентами. При этом, необходимо добавить собственную строку вызова своей функции таким образом, чтобы в таблице номера студентов шли по порядку (сначала 01, затем 02, потом 03 и т.д.). После того, как сделаны правки необходимо сохранить изменения, выйти из редактора, подтвердив что файл исправлен.

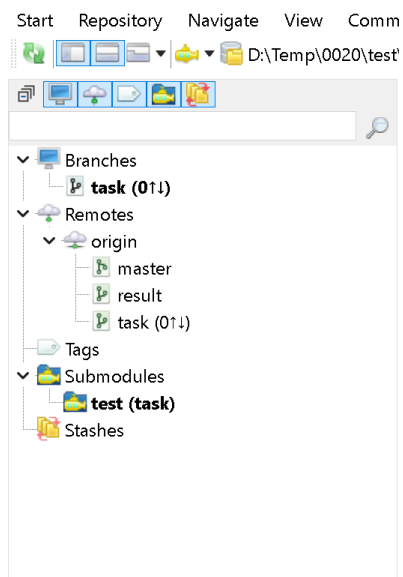
7. Перед тем, как зафиксировать изменения необходимо проверить компиляцию и работоспособность программы, после этого сделать Commit (!!! При разрешении конфликта нельзя изменять текст комментария при коммите)
8. Необходимо снова залить изменения ветки на сервер и предупредить об этом преподавателя
9. После принятия «Merge Request» преподавателем задание считается выполненным
10. Если снова возникнет конфликт, но необходимо заново выполнить все шаги данного раздела

Приложения

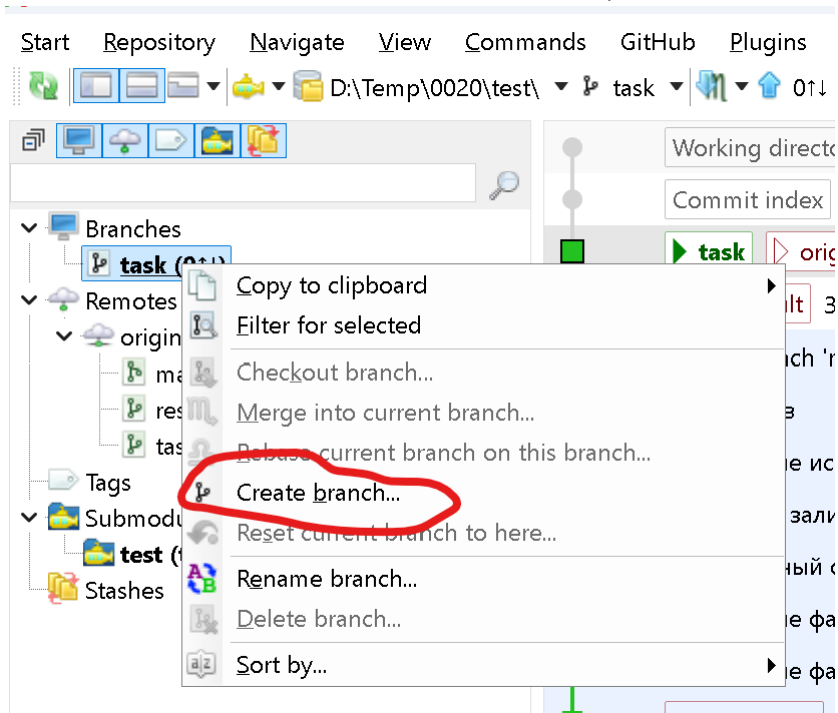
Пример разрешения конфликта

Студент Пётр Петров выполняет свою часть задания

1. Клонировал репозиторий и переключился на ветку task



2. Теперь необходимо создать собственную ветку для выполнения задания. Для этого нужно вызвать контекстное меню для ветки task и выбрать «Create Branch»



Назвать ветку, в соответствии с требованиями задания

Create branch

Branch name: 12Petrov

Create branch at this revision: 10de7437

☒ Checkout after create

10de7437

Актуализация файла .gitignore

Author: vstolchnev

Commit date: 6 minutes ago (24.02.2025 7:23:44)

Branch(es): task

Tag(s): n/a

Orphan

☐ Create orphan ☒ Clear working directory and index

Create branch

3. Открыть в VSCode папку с файлом задания из репозитория. Доработать код по требованию задания

```
using namespace std;

struct People
{
    int id {};
    wstring name;
    int height;
};

People create12PetrovInfo()
{
    People student;

    student.id = 12;
    student.name = L"Petrov";
    student.height = 188;

    return student;
}

void showPeoples(const vector<People>& listOfPeoples)
{
    const int columnNumLength = 4;
```

```
61
62 int main()
63 {
64     setmode(_fileno(stdout), _O_U8TEXT);
65     vector<People> listOfPeoples;
66     listOfPeoples.push_back(create12PetrovInfo());
67     showPeoples(listOfPeoples);
68 }
```

4. Проверить что программа выполняется и компилируется. Исправить ошибки, при необходимости

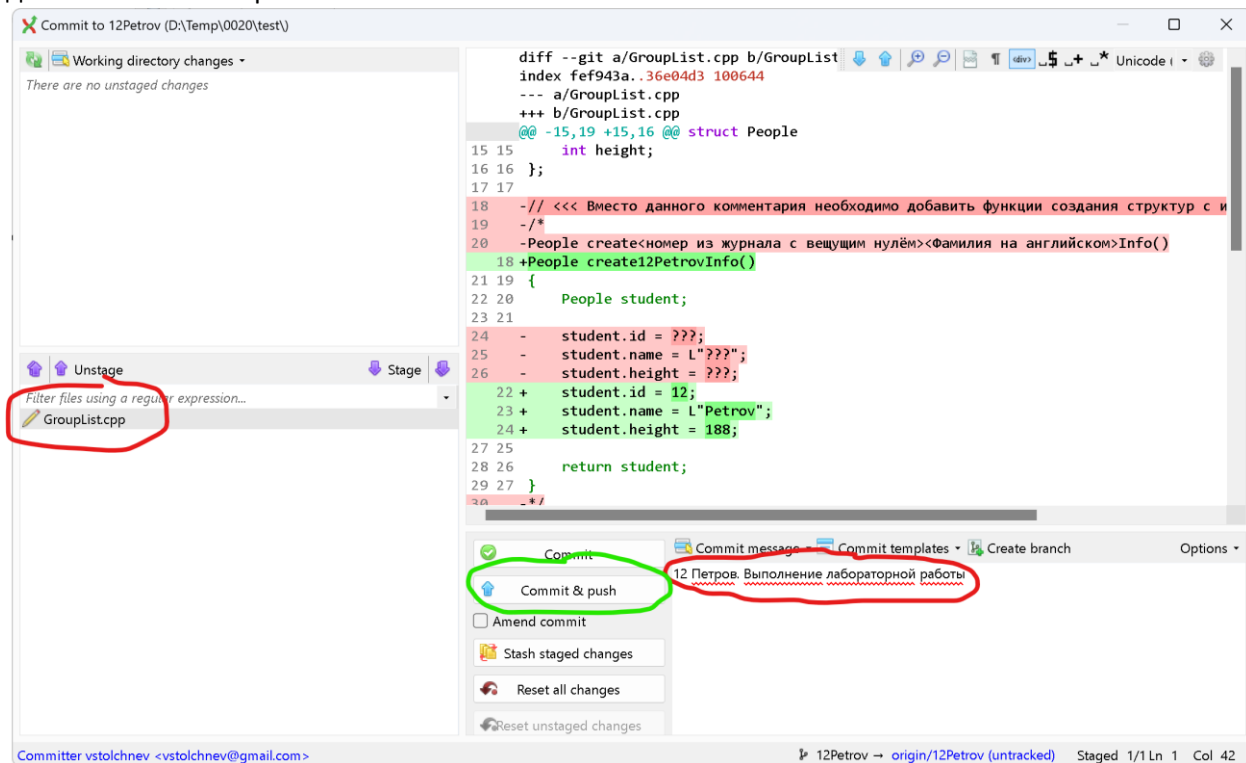
```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS D:\Temp\0020\test> & 'c:\Users\User\.vscode\extensions\ms-vscode.cp
dowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-qcbuxa14.nmh' '--
--stderr=Microsoft-MIEngine-Error-1i3mgat2.ymu' '--pid=Microsoft-MIEngi
w64\bin\gdb.exe' '--interpreter=mi'

+=====+
| № | ФИО | Рост |
+=====+
| 12 | Petrov | 188 |
+-----+

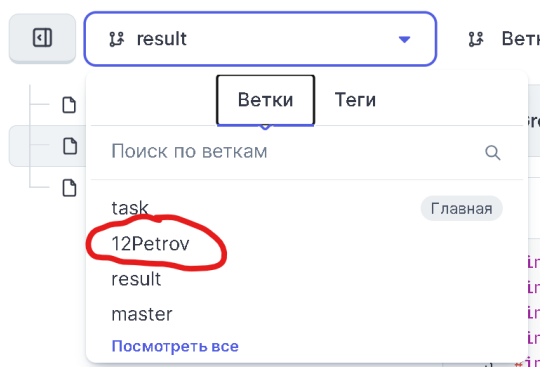
○ PS D:\Temp\0020\test>
```

5. Зафиксировать изменения и залить на сервер. При внесении изменений обязательно добавить комментарий



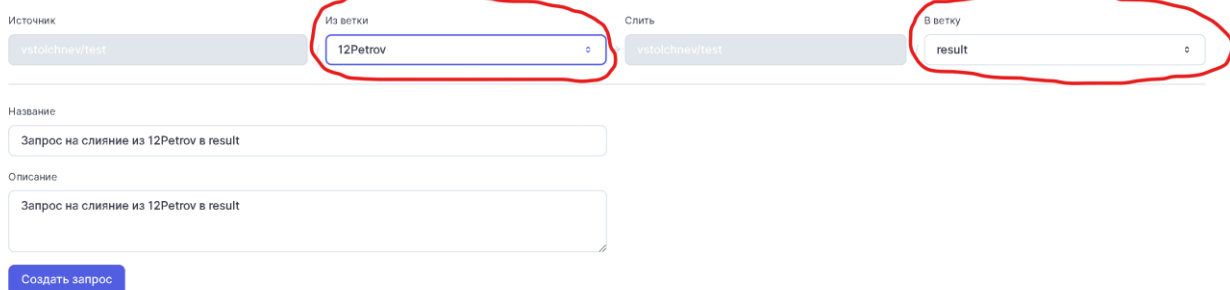
Если после выполнения команды Push отображается диалоговое окно с вопросом по связке локальной ветке с залитой на удалённый репозиторий, то нужно ответить положительно.

6. Перейти на сайт gitverse, обновить страницу с проектом, при необходимости. На главной странице проекта в выпадающем списке должна появиться созданная ветка



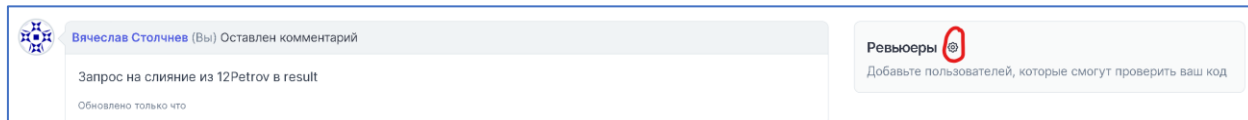
7. На сайте gitverse перейти в раздел «Запросы» и нажать на кнопку «Новый запрос»
В выпадающем списке «Из ветки» нужно выбрать созданную ветку (в данном примере это «12Petrov»). В выпадающем списке «В ветку» выбрать «result»

Новый запрос на слияние



Нажать на кнопку «Создать запрос»

8. После создания запроса на слияние, сразу необходимо добавить преподавателя в «Ревьюеры»



Сделать это можно нажав на кнопку в виде шестерёнки рядом с текстом «Ревьюеры».

9. На странице с созданным запросом будет отображена информация о том, что возник конфликт. Если про конфликт ничего не показывается, то, возможно, исходная ветка была создана не из task. Нужно перепроверить. При необходимости удалить созданную ветку

(локально и на сервере) и сделать заново.

04312b71faf 12 Петров. Выполнение лабораторной работы

только что

 **В запросе на слияние есть конфликты**
Решите конфликты, чтобы произвести слияние

Вы должны разрешить конфликты путем ручного слияния **result** с этой веткой. После устранения конфликтов вы сможете произвести запрос на слияние.

Шаг 1: Проверьте исходную ветвь и объедините изменения из целевой ветви. Разрешите конфликты:

```
git checkout 12Petrov
git pull origin result
```

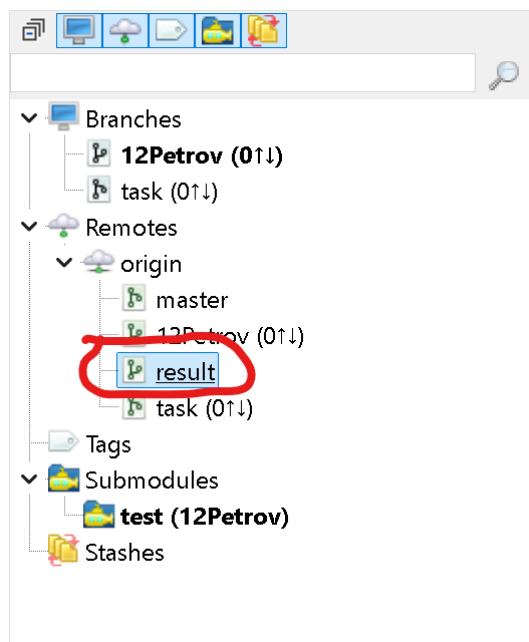
Шаг 2: После разрешения конфликтов слияния соответствующим образом оформить изменения, зафиксировать их и выполнить push.:

```
git commit
git push origin HEAD
```

Шаг 3: Внесите изменения:

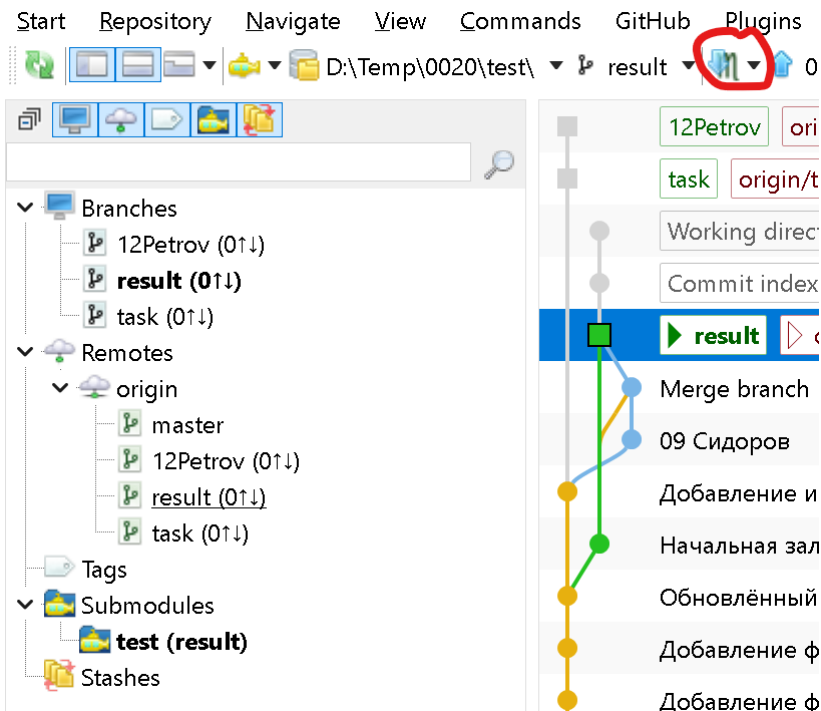
```
git push -u origin result
```

10. Для устранения конфликта необходимо вернуться в локальный репозиторий и переключиться на ветку result с сервера. Быстро переключиться на ветку можно двойным щелчком мыши.

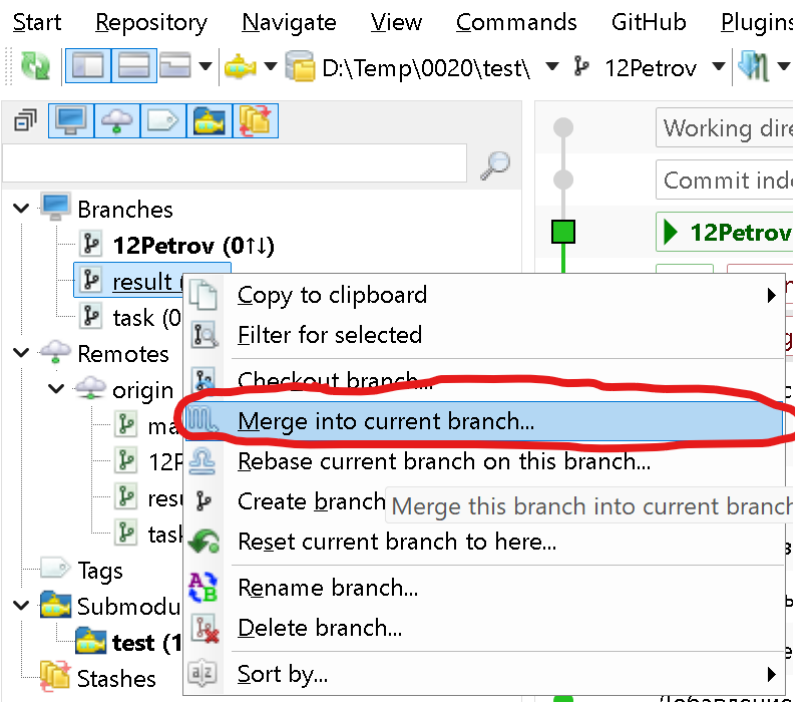


11. После переключения на ветку, она отобразится «жирным» шрифтом в дереве веток. Для самоконтроля нужно дополнительно актуализировать содержимое ветки, выполнив

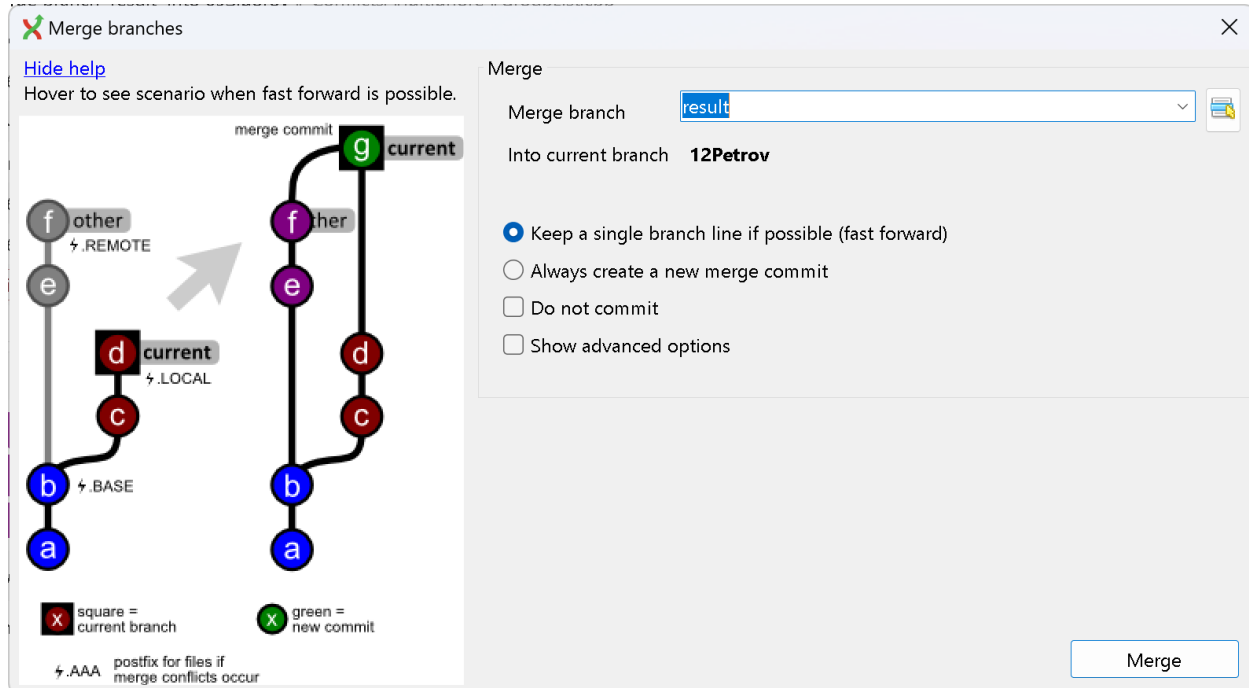
команду Pull



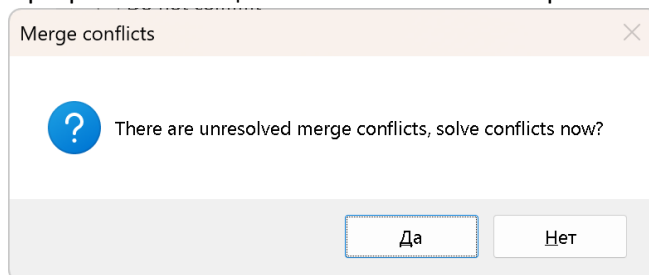
12. Затем, двойным щелчком мыши нужно переключиться на собственную ветку и объединить её с веткой result. Для этого, нужно вызвать контекстное меню для локальной ветки result и выбрать пункт «Merge into current branch»



13. В появившемся диалоговом окне настройки объединения веток ничего менять не нужно, просто нажать кнопку «Merge»

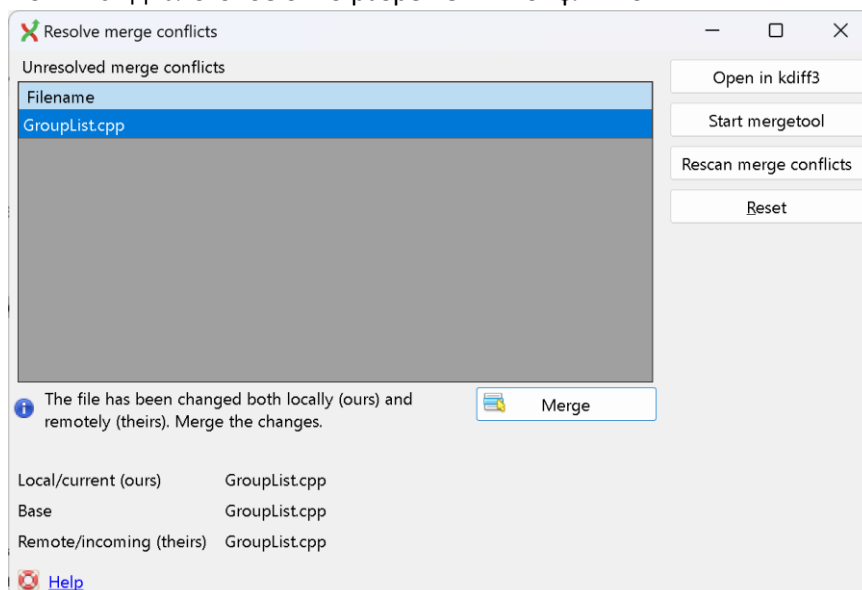


14. Программа сообщит о возникновении конфликта и предложит его разрешить



Необходимо выбрать «Да»

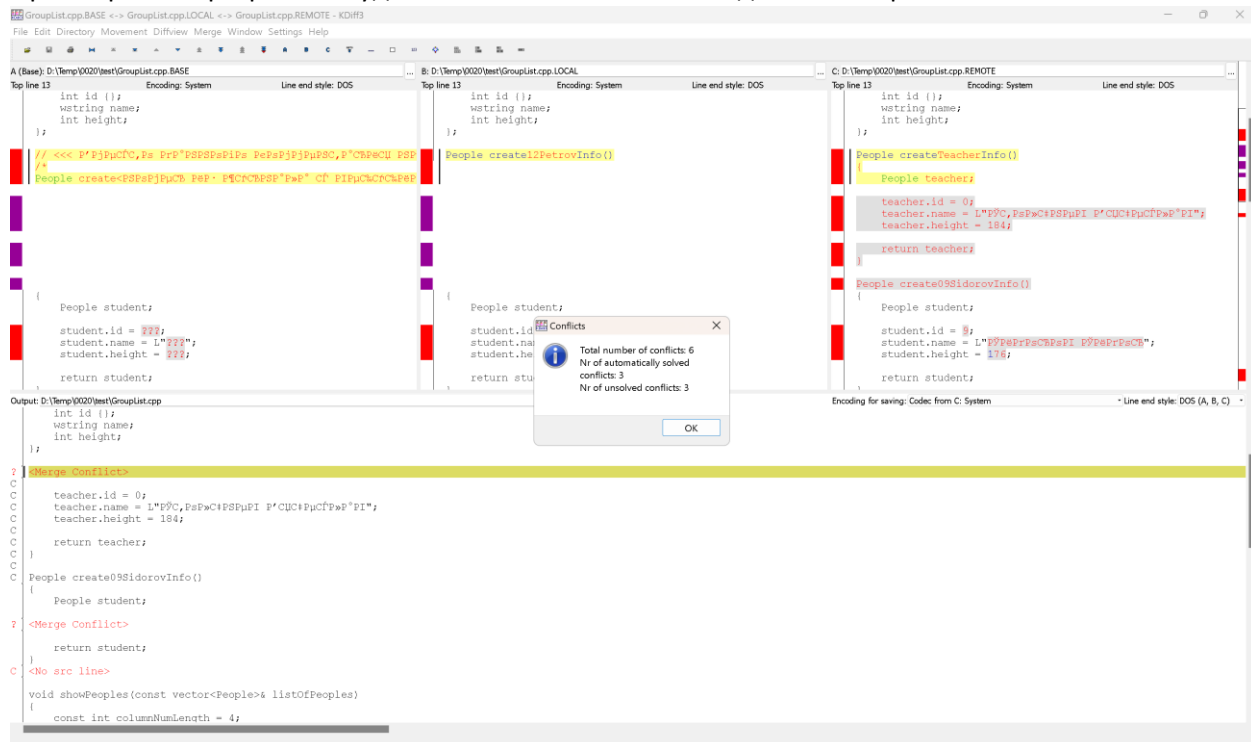
15. Появится диалоговое окно разрешения конфликтов



Если всё делалось правильно, то должен быть один конфликтный файл с исходным кодом программы.

Также, если в качестве программы разрешения конфликтов настраивалась kdiff3, то будет предложено открыть её для разрешения конфликтов, что и нужно сделать «Open in kdiff3»

16. При открытии программы будет показано количество найденных конфликтов.



Если кодировка отображается некорректно, то их можно изменить в настройках kdiff3, но для применения изменений придётся заново открыть редактор (можно просто выйти без сохранения изменений и заново открыть).

Окно редактора состоит из четырёх частей. Верхние условно называются «А», «В», «С».

Они показывают

«А» - содержимое конфликтного файла до изменений

«В» - изменения в ветке куда делается объединение (для данного примера это ветка 12Petrov)

«С» - изменения в ветке из которой делается объединение (для данного примера это ветка result)

Задача объединения — получить корректное содержимое файла с учётом всех изменений (в реальной жизни часть изменений отклоняется/удаляется и это тоже нормально). Для данного примера необходимо получить целевое содержимое файла с учётом изменений всех студентов и преподавателя. Для этого, сначала, предлагается принять все изменения из ветки «result» (окно «С»). Необходимо перейти на первую запись с неразрешённым конфликтом, для этого нужно нажимать на кнопку из рисунка ниже пока она не станет «серым» цветом:

erge window Settings Help



После этого нажать на кнопку «С» - в итоговый файл пропишутся изменения из ветки result, затем нажать на кнопку перехода к следующему неразрешённому конфликту и снова на «С»

erge window Settings Help



Данные действия проделать пока не будут перебраны все найденные конфликты (обе кнопки перехода по неразрешённым конфликтам станут «серыми»)



```
Output: D:\Temp\0020\test\GroupList.cpp
```

```
C  
C People create09SidorovInfo()  
C {  
C     People student;  
  
C     student.id = 9;  
C     student.name = L"Сидоров Сидор";  
C     student.height = 176;  
  
    return student;  
}  
m  
m People create12PetrovInfo()  
m {  
m     People student;  
  
m     student.id = 12;  
m     student.name = L"Petrov";  
m     student.height = 188;  
  
m     return student;  
m }  
  
void showPeoples(const vector<People>& listOfPeoples)  
{  
    const int columnNumLength = 4;  
    const int columnNameLenght = 50;
```

Ready.

```
int main()
{
    _setmode(_fileno(stdout), _O_U8TEXT);
    vector<People> listOfPeoples;
    listOfPeoples.push_back(createTeacherInfo());
    listOfPeoples.push_back(createO9SidorovInfo());
    listOfPeoples.push_back(createI2PetrovInfo());
    showPeoples(listOfPeoples);
}
```

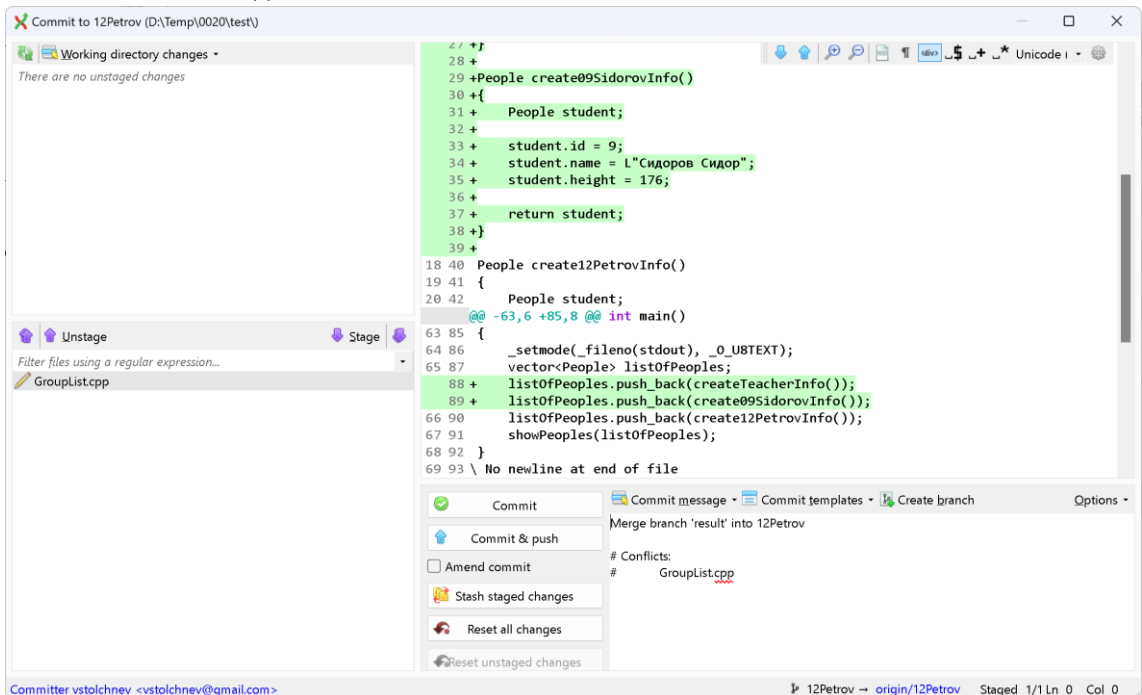
17. После внесения изменений нужно нажать на кнопку «Сохранить» и закрыть редактор Kdiff. В GitExtensions покажется диалоговое окно с предложением сделать Commit и зафиксировать изменения. Сразу это делать не нужно. Необходимо открыть VSCode и проверить что программа компилируется и правильно работает:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

--stderr=Microsoft-MIEngine-Error-kay5ndtr.kak' '--pid=Microsoft-MIE
w64\bin\gdb.exe' '--interpreter=mi'

+====+=====+=====+=====+=====+=====+=====+=====+=====+
| № | ФИО | | Рост |
+====+=====+=====+=====+=====+=====+=====+=====+=====+
| 0 | Столчев Вячеслав | | 184 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 9 | Сидоров Сидор | | 176 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 12 | Petrov | | 188 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

После этого можно делать Commit и Push в GitExtensions



!!! Изменять текст для Commit не нужно

18. В gitverse новый запрос на слияние можно не создавать. Нужно зайти на страницу созданного ранее запроса (либо обновить в браузере, если она уже была открыта).

Конфликт исчезнет:

Запрос на слияние из 12Petrov в result

Обновлено только что

Вячеслав Столчнев (Вы)

Добавлены коммиты

18de74371a1

Актуализация файла .gitignore

64312b71faf

12 Петров. Выполнение лабораторной работы

час назад

Вячеслав Столчнев (Вы)

Добавлены коммиты

ac5bd796572

Merge branch 'result' into 12Petrov # Conflicts: # GroupList.cpp

только что

У ветки 12Petrov нет конфликтов с веткой result

Объединение веток можно выполнить безопасно

Название

Запрос на слияние '12Petrov' (#5) из 12Petrov в result

Описание

☐ Удалить ветку «12Petrov»

19. Дождаться когда преподаватель пример запрос на слияние. Если запрос на слияние не первый в очереди, то может получиться, что сначала примут запрос на слияние другого студента и снова возникнет конфликт. Если такое произошло, то конфликт нужно разрешать заново.